

Adversarial Attacks, Formal Verification and Certified Training of a Fully Connected MNIST Classifier

Final Project — Trustworthy Artificial Intelligence Systems

Abstract

This report studies the complete robustness life-cycle of a small fully connected ReLU classifier ($784 \rightarrow 64 \rightarrow 32 \rightarrow 10$) trained on MNIST: we (i) train it conventionally for two epochs, (ii) break it with the Fast Gradient Sign Method, (iii) prove its fragility formally with three sound abstract domains (interval, zonotope/DeepZono, DeepPoly/CROWN) and with a *complete* mixed-integer programming verifier, (iv) retrain it with Interval Bound Propagation using a worst-case-logit loss whose design we derive and justify in detail, and (v,vi) repeat the attack and the verification on the hardened model. At $\varepsilon^* = 0.1$ in the ℓ_∞ norm, FGSM reduces the baseline from 95.77% to 8.92% accuracy and every abstract domain certifies 0%; after IBP training the same attack leaves 88.02% accuracy and a *complete* verifier certifies exactly 82%, while clean accuracy is essentially unchanged (95.63%). We further show that certified training does not merely make the network robust — it makes it *verifiable*: the number of unstable ReLUs per input falls from 81.7 to 12.8 out of 96, the width of the output bounds falls by a factor of 33, and the cheap interval domain becomes exactly as precise as the expensive ones, with zero incompleteness gap against the complete MILP oracle.

1 Introduction and scope

A neural network that is accurate on a test set may still be catastrophically wrong on inputs that are visually indistinguishable from test points. The standard formalisation of this concern is *local robustness*: for a classifier $f : \mathbb{R}^{784} \rightarrow \mathbb{R}^{10}$, a labelled input (x, c) and a perturbation budget ε , we ask whether

$$\forall x' \in \mathcal{B}_\varepsilon(x) : \arg \max_k f_k(x') = c, \quad \mathcal{B}_\varepsilon(x) = \{x' : \|x' - x\|_\infty \leq \varepsilon\} \cap [0, 1]^{784}. \quad (1)$$

The intersection with $[0, 1]^{784}$ matters: pixels are bounded, so the true input region is a clipped box, and using it makes both attacks and proofs tighter (and mutually comparable).

Three qualitatively different tools address (1). **Attacks** (FGSM, PGD) search for a violating x' ; a success is a *proof of non-robustness*, but a failure proves nothing. **Incomplete verifiers** over-approximate the reachable set; a success is a *proof of robustness*, but a failure is inconclusive. **Complete verifiers** decide the question exactly, at exponential worst-case cost. This report uses all three, so that every claim we make is of the strongest kind available.

The assignment allocates one verification tool per student (ERAN with the `deepzono/deeppoly` domains, or CROWN via `auto_LiRPA`). Since both tools implement the same two abstract domains, and since the analysis environment available for this submission had neither a GPU nor a working PyTorch/ELINA installation (Section 2), we re-implemented both domains from first principles in NumPy and validated them against (a) exhaustive sampling, (b) each other, and (c) an exact MILP oracle. Ready-to-run artefacts for the official tools — an ONNX export for ERAN and a `.pth` converter plus an `auto_LiRPA` driver script for CROWN — are included with the code and are numerically identical to the analysed network to within 1.2×10^{-5} .

2 Experimental setup

Environment. Ubuntu 24.04 (kernel 6.18), single CPU core, no GPU. Python 3.12.3 with NumPy 2.4.4, SciPy 1.17.1 (its `optimize.milp` interface to the HiGHS solver is used for complete

verification), Matplotlib 3.10.8, ONNX 1.22.0 and ONNX Runtime 1.24.4. No deep-learning framework is required: the network, its backward pass, the attacks, the interval propagation and all four verifiers are implemented directly in NumPy (`code/tas_lib.py`, `code/verifiers.py`). Every experiment is seeded (`seed = 1405`) and the whole pipeline runs end-to-end in under seven minutes.

Data. The original MNIST IDX files (60 000 training and 10 000 test images) are read directly; MD5 checksums match the canonical files. Pixels are scaled to $[0, 1]$ and images are flattened to 784 values. No mean/variance normalisation is applied, deliberately: keeping the network input in the same space as the raw image means that ε is directly a pixel budget and that the clipping in (1) is exact.

3 Step 1 — Architecture and standard training

The required architecture is a three-layer fully connected ReLU network, summarised in Table 1. Layer 3 emits the ten raw logits, i.e. the “one-hot-encoded” output of the assignment; the softmax is folded into the loss.

Layer	Operation	Input	Output	Parameters
1	Linear $\rightarrow$ ReLU	784	64	50 240
2	Linear $\rightarrow$ ReLU	64	32	2 080
3	Linear	32	10	330
total				52 650

Table 1: The classifier under study. It contains $64 + 32 = 96$ ReLU units, which is what makes exact (complete) verification computationally feasible.

Weights use He initialisation, biases start at zero, and the network is trained with Adam ($\eta = 10^{-3}$, batch size 128) for **two epochs** only, as instructed — long enough to be a credible classifier, short enough that adversarial attacks remain clearly effective. The result is 96.09% train / **95.77%** test accuracy in 1.1 s (Figure 1a).

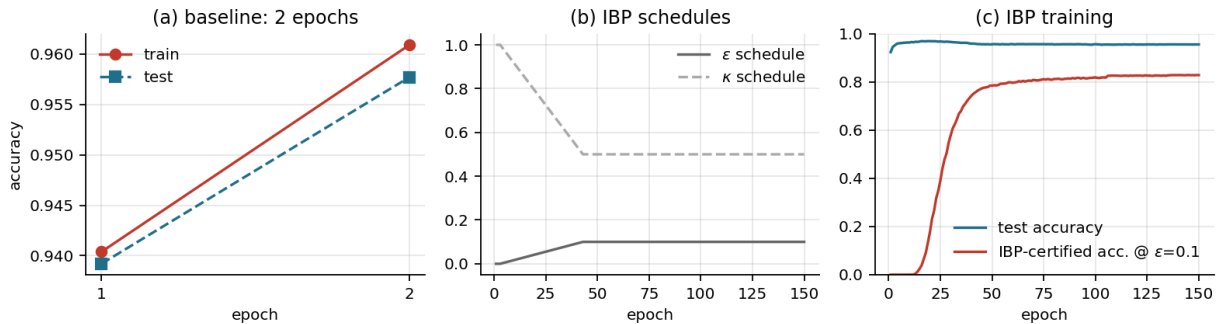


Figure 1: (a) Baseline training. (b) The ε and κ schedules used for certified training in Step 4. (c) Standard and IBP-certified test accuracy during certified training; note that certified accuracy is zero until the ε -ramp begins and then rises smoothly, while clean accuracy is barely affected.

4 Step 2 — Attacking the model with FGSM

4.1 Method

FGSM [1] takes a single step of size ε in the direction that maximises the training loss, using the sign of the input gradient because that is the steepest-ascent direction under an ℓ_∞ constraint:

$$x_{\text{adv}} = \Pi_{[0,1]} \left(x + \varepsilon \text{sign}(\nabla_x \mathcal{L}(f(x), c)) \right). \quad (2)$$

It is a *linearisation* attack: it assumes the loss is approximately affine over $\mathcal{B}_\varepsilon(x)$. It is therefore cheap but not optimal, and we also report PGD [2] — iterated FGSM with projection and random restarts — as a strictly stronger reference. Any input on which either attack succeeds is definitively non-robust.

4.2 Results and the choice of ε^*

ε	0.01	0.02	0.03	0.05	0.075	0.10	0.15	0.20	0.30
accuracy under FGSM	92.34	87.01	79.13	54.91	24.69	8.92	0.84	0.10	0.00
newly fooled (% of test set)	3.43	8.76	16.64	40.86	71.08	86.85	94.93	95.67	95.77

Table 2: FGSM against the baseline model (clean accuracy 95.77%). “Newly fooled” counts test images that were classified correctly and became incorrect.

Following the phrasing of the assignment, a representative statement of the result is: *adding noise of magnitude $\varepsilon = 0.05$ causes misclassification of 40.86% of the test samples that were previously correct; at $\varepsilon = 0.1$ this rises to 86.85%.* We adopt

$$\varepsilon^* = 0.1$$

as the operating point for Steps 3–6. Three reasons: (i) it is the canonical ℓ_∞ benchmark for MNIST, so our certified numbers are comparable with the literature; (ii) it destroys the baseline unambiguously (95.77% $\rightarrow$ 8.92% under FGSM, $\rightarrow$ 3.13% under PGD-50), which makes Step 3’s proof of weakness non-controversial; and (iii) it is large enough that certified training is genuinely challenging rather than trivial.

At ε^* the attack fools **8685** test images (90.69% of all correctly classified ones). Their indices, clean images, adversarial images, true labels and adversarial predictions are stored in `results/fooled_samples.npz` and re-used in Step 5, as required. Figure 2 shows a representative selection: the perturbations are structured (they trace the stroke boundaries) but semantically meaningless to a human observer.

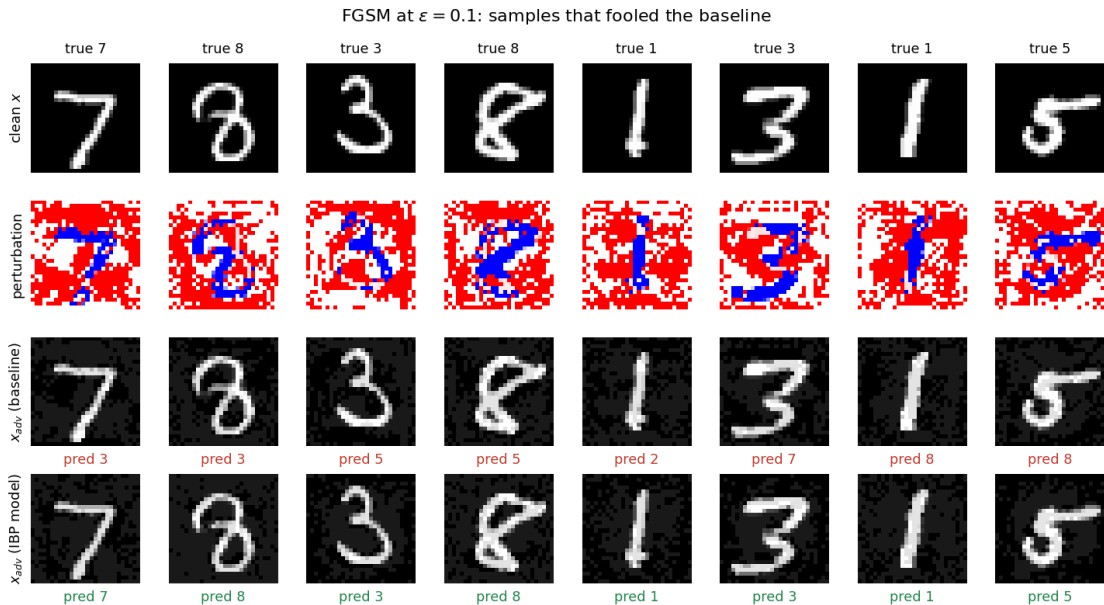


Figure 2: Eight of the 8685 test images that FGSM flips on the baseline model at $\varepsilon = 0.1$. Row 1: clean input. Row 2: the perturbation $\varepsilon \text{sign}(\nabla_x \mathcal{L})$ (red/blue = $\pm\varepsilon$). Row 3: the adversarial input and the baseline’s (wrong) prediction. Row 4: the same attack re-crafted against the IBP-trained model of Step 4 — green labels are recovered predictions (Step 5).

5 Step 3 — Proving the weakness formally

5.1 The specification

Property (1) is equivalent to positivity of the worst-case margin

$$m^*(x, c) = \min_{j \neq c} \min_{x' \in \mathcal{B}_\varepsilon(x)} (z_c(x') - z_j(x')) > 0, \quad (3)$$

where $z = f(x')$ are the logits. Every verifier below computes a *lower bound* $\underline{m} \leq m^*$; if $\underline{m} > 0$ the input is certified robust. Only the MILP verifier computes m^* exactly.

5.2 Three sound abstract domains

(a) Interval / IBP (the box domain). Bounds are propagated element-wise. For an affine layer $z = aW + b$ we use the numerically stable midpoint–radius form, which is *exact* for a box input:

$$\mu = \frac{a+\bar{a}}{2}, \quad r = \frac{\bar{a}-a}{2}, \quad \underline{z} = \mu W + b - r|W|, \quad \bar{z} = \mu W + b + r|W|. \quad (4)$$

ReLU is monotone, so $[z, \bar{z}] \mapsto [\text{relu}(\underline{z}), \text{relu}(\bar{z})]$ is exact too. The imprecision comes entirely from *losing the correlations between neurons*: after the first layer the box treats all 64 activations as independent, whereas they are all functions of the same 784 inputs.

(b) Zonotope / DeepZono [3]. A zonotope $\hat{a} = a_0 + \sum_i g_i \epsilon_i$, $\epsilon_i \in [-1, 1]$, keeps those correlations: affine layers act exactly on the generators g_i . Only ReLU loses information. For a *crossing* unit ($\underline{z} < 0 < \bar{z}$) the area-minimal transformer is

$$\hat{y} = \lambda \hat{z} + \mu + \mu \epsilon_{\text{new}}, \quad \lambda = \frac{\bar{z}}{\bar{z} - \underline{z}}, \quad \mu = -\frac{\lambda \underline{z}}{2}, \quad (5)$$

which introduces one fresh noise symbol per unstable neuron.

(c) DeepPoly / CROWN [4, 5]. Each ReLU is bounded by two affine functions of its own pre-activation,

$$\lambda z \leq \text{relu}(z) \leq \frac{\bar{z}}{\bar{z} - \underline{z}} (z - \underline{z}), \quad \lambda \in [0, 1], \quad (6)$$

and the resulting linear expressions are *back-substituted* all the way to the input layer before being concretised over the input box. Writing the objective as $A^{(L)} z^{(L)} + d$, the recursion through a ReLU splits on the sign of each coefficient (positive coefficients use the lower relaxation, negative ones the upper), and through an affine layer it is exact. We use the area-minimising heuristic $\lambda = \mathbb{I}[\bar{z} \geq -\underline{z}]$, which is exactly ERAN’s DeepPoly rule and CROWN’s adaptive rule; with this choice *the two tools compute identical bounds on a feed-forward ReLU network*, which is why re-implementing one of them suffices.

Remark 1 (Intersection with the box). *Because the lower relaxation in (6) can dip below the trivial bound $\text{relu}(z) \geq 0$, neither the zonotope nor DeepPoly dominates the box domain a priori. Both ERAN and `auto_LiRPA` therefore keep the tighter of the two at every layer, and so do we. Without this intersection we measured DeepPoly bounds that were occasionally worse than plain intervals.*

Remark 2 (DeepZono vs. DeepPoly). *Our randomised cross-checks confirm the known theoretical result that the two domains are incomparable: on random networks each is sometimes tighter. On the real network DeepPoly wins on average (Table 3).*

5.3 A complete verifier: mixed-integer programming

Sound-but-incomplete answers leave “unknown” cases, so we additionally encode the exact problem as a MILP [7]. Each unstable ReLU $a = \text{relu}(z)$ with $\underline{z} < 0 < \bar{z}$ becomes, with $\delta \in \{0, 1\}$,

$$a \geq 0, \quad a \geq z, \quad a \leq \bar{z}\delta, \quad a \leq z - \underline{z}(1 - \delta), \quad (7)$$

which is an exact “big-M” encoding; stable units are substituted away. The tighter the pre-activation bounds $\underline{z}, \bar{z}$, the tighter the relaxation, so we feed the DeepPoly bounds into the encoding. Rather than *minimising* the margin we solve a *feasibility* problem per target class j — “does there exist $x' \in \mathcal{B}_\varepsilon(x)$ with $z_j \geq z_c$?” — because the solver may then stop as soon as infeasibility is proven instead of closing an optimality gap; on our instances this was roughly an order of magnitude faster, and we verified that both formulations always agree.

5.4 Validation of the implementation

Before trusting any number we checked, on random networks with 8 inputs and 4 classes, that (i) all three abstract domains give bounds below the exact MILP optimum, (ii) the MILP optimum lies below the best margin found by 150 000 random samples per input, and (iii) the feasibility and optimisation MILP formulations agree on every instance. All checks passed.

5.5 Results

ε	certified accuracy (%) on the first 100 test images					
	0.01	0.02	0.03	0.05	0.075	0.10
<i>baseline model</i> (clean accuracy on this subset: 99%)						
PGD-50 (upper bound on robustness)	98	91	78	40	8	1
Interval / IBP	0	0	0	0	0	0
DeepZono	95	82	39	5	0	0
DeepPoly / CROWN	95	83	44	5	0	0
<i>IBP-trained model</i> (Step 4; clean accuracy on this subset: 99%)						
PGD-50 (upper bound on robustness)	98	98	97	96	93	87
Interval / IBP	98	98	97	95	91	82
DeepZono	98	98	97	95	91	82
DeepPoly / CROWN	98	98	97	95	91	82

Table 3: Certified accuracy versus perturbation budget. PGD gives an upper bound on the true robust accuracy; the abstract domains give lower bounds. The true value lies between them.

At $\varepsilon^* = 0.1$ the verdict on the baseline is unambiguous (Table 4): **no test input at all can be certified by any domain**, 98 of 100 are *definitively falsified* by an explicit counterexample, one is already misclassified without any perturbation, and a single instance (index 82) remained undecided within the 30 s MILP budget. For that instance a very strong targeted PGD attack (30 restarts $\times$ 400 steps $\times$ 9 target classes) could not push the margin below +2.41, so it is most likely robust; we report it honestly as TIMEOUT rather than claiming a proof. Since a robust point contributes at most one percentage point, the conclusion — certified accuracy $\in [0\%, 1\%]$ — is unaffected.

The ε -sweep is more informative than the single operating point. At $\varepsilon = 0.03$ PGD leaves 78% accuracy while DeepPoly certifies only 44%: a 34-point *incompleteness gap*. This is the characteristic signature of a normally trained network — it is not that the network is non-robust there, it is that the abstraction is too coarse to prove it. Note also that the interval domain certifies *nothing at all*, even at $\varepsilon = 0.01$: after two affine layers the accumulated radius $r|W_1||W_2|$ already dwarfs the logit gaps.

	baseline	IBP-trained
clean accuracy (100-image subset)	99	99
certified robust — interval / DeepZono / DeepPoly	0 / 0 / 0	82 / 82 / 82
complete (MILP) robust	0	82
falsified (explicit counterexample)	98	17
of which found by PGD / by MILP	98 / 0	13 / 4
misclassified without perturbation	1	1
undecided (30s MILP timeout)	1	0
unstable ReLUs per input (mean, of 96)	81.7	12.8
layer 1 (of 64) / layer 2 (of 32)	52.0 / 29.7	7.4 / 5.5
median width of the 10 output bounds — IBP	124.8	3.76
median width of the 10 output bounds — DeepPoly	36.5	3.68
runtime, 100 inputs — interval / DeepZono / DeepPoly	0.002 / 0.08 / 0.48 s	0.002 / 0.08 / 0.46 s
runtime, MILP phase	30.5 s (1 input)	3.5 s (4 inputs)

Table 4: Complete verification results at $\varepsilon^* = 0.1$ (Steps 3 and 6).

6 Step 4 — Certified training with IBP

6.1 From bounds to a loss

Ordinary training minimises $\mathcal{L}(f(x), c)$ at the single point x . Adversarial training replaces x by an attacked $\tilde{x}$, which improves empirical robustness but certifies nothing — it optimises against *one* point of $\mathcal{B}_\varepsilon(x)$, chosen by a heuristic. IBP training instead pushes the *whole box* through the network using (4) and optimises the worst case over the resulting bounds. This is the crucial conceptual step: the quantity being minimised is the same quantity a verifier will later try to certify.

Propagating $\mathcal{B}_\varepsilon(x)$ yields per-logit bounds $\underline{z}_k \leq z_k(x') \leq \bar{z}_k$ valid for *all* $x' \in \mathcal{B}_\varepsilon(x)$. Now consider the specification (3). For the true class we need the smallest value it can take, and for every wrong class the largest:

$$\hat{z}_k = \begin{cases} \underline{z}_c = z_{\min}[c], & k = c \quad (\text{the attacker's best attempt to } \textit{lower} \text{ the correct score}), \\ \bar{z}_k = z_{\max}[k], & k \neq c \quad (\text{the attacker's best attempt to } \textit{raise} \text{ a wrong score}). \end{cases} \quad (8)$$

This is exactly the construction requested in the assignment. Compactly, $\hat{z} = \mu^{(3)} + s \odot r^{(3)}$ with $s_c = -1$ and $s_j = +1$ otherwise, where $\mu^{(3)}, r^{(3)}$ are the midpoint and radius of the logit box.

Proposition 1 (Soundness). *If $\arg \max_k \hat{z}_k = c$ then x satisfies (1).*

Proof. For any $x' \in \mathcal{B}_\varepsilon(x)$ and any $j \neq c$ we have $z_c(x') - z_j(x') \geq \underline{z}_c - \bar{z}_j = \hat{z}_c - \hat{z}_j$. If $\hat{z}_c$ is the strict maximum of $\hat{z}$, the right-hand side is positive for every j , hence so is the left-hand side. $\square$

Remark 3 (Why this is conservative, not exact). *The ten entries of $\hat{z}$ are attained by ten different adversarial inputs in general; no single x' realises them simultaneously. $\hat{z}$ is therefore a pessimistic “virtual adversary” that is at least as strong as any real one. This is precisely what makes the resulting certificate valid — and also why certified accuracy is a lower bound on true robust accuracy.*

6.2 Why cross-entropy on $\hat{z}$

Given $\hat{z}$, the loss is

$$\mathcal{L}_{\text{rob}}(x, c) = \text{CE}(\hat{z}, c) = -\log \frac{e^{\hat{z}_c}}{\sum_k e^{\hat{z}_k}} = \log \left(1 + \sum_{j \neq c} e^{-(\hat{z}_c - \hat{z}_j)} \right). \quad (9)$$

Four properties motivate this specific choice over the obvious alternatives (e.g. a hinge $\sum_j \max(0, \gamma - (\hat{z}_c - \hat{z}_j))$ or a min-margin loss):

- (i) **It upper-bounds the verified error.** The rightmost form in (9) shows $\mathcal{L}_{\text{rob}}$ is a decreasing, convex, smooth function of the certified margins $\hat{m}_j = \hat{z}_c - \hat{z}_j = z_c - \bar{z}_j$. Moreover $\mathcal{L}_{\text{rob}} \geq \log 2 \cdot \mathbb{K}[\exists j : \hat{m}_j \leq 0]$, so minimising it minimises a smooth surrogate of the quantity we actually care about — the fraction of inputs a verifier will fail to certify.
- (ii) **It degenerates gracefully to ordinary training.** At $\varepsilon = 0$ the box collapses, $r^{(3)} = 0$, and $\hat{z} = z$, so (9) becomes the standard cross-entropy. This is what allows a single objective and a continuous ε ramp (Section 6.3); a hinge loss would introduce a second, unrelated hyper-parameter γ whose correct scale changes as ε grows.
- (iii) **It allocates gradient adaptively across the nine wrong classes.** $\partial \mathcal{L}_{\text{rob}} / \partial \hat{z}_j = \text{softmax}(\hat{z})_j$, so the currently most threatening class receives most of the gradient, and classes that are already far away receive almost none. A sum of hinges treats all nine equally and wastes capacity; a pure $\min_j$ loss is non-smooth and gives gradient to only one class per step, which we found trains noticeably slower.
- (iv) **It cannot be satisfied trivially.** Because $\hat{z}_c - \hat{z}_j = z_c - \bar{z}_j$ depends on the bound *radii*, the only way to reduce (9) is to shrink $r^{(3)}$ — i.e. to make the network genuinely less sensitive over the box — or to increase the nominal logit gap. Both are desirable.

6.3 The mixed objective and the schedules

Training on $\mathcal{L}_{\text{rob}}$ alone, at full ε , from a random initialisation, fails: we measured a median output-bound width of ~ 125 at initialisation, so $\hat{z}$ is dominated by the radius term, every class looks equally bad, and the fastest way to reduce the loss is to drive all weights to zero — the network collapses to a constant classifier. Following Gowal et al. [8] we therefore use

$$\mathcal{L} = \kappa \text{CE}(f(x), c) + (1 - \kappa) \text{CE}(\hat{z}(\varepsilon_t), c), \quad (10)$$

with ε_t ramped linearly $0 \rightarrow \varepsilon^*$ and κ annealed $1 \rightarrow 0.5$ over the same window (3 warm-up epochs, then a 40-epoch ramp, then 107 epochs at the final setting; Adam, $\eta = 5 \times 10^{-4}$ decayed $\times 0.2$ at 70% and 90% of training). The κ term preserves clean accuracy and keeps the logits well-scaled; the ε ramp means the network is never asked to satisfy a constraint it cannot yet meet. Figure 1(b,c) shows the effect: certified accuracy tracks the ramp smoothly and clean accuracy never drops.

6.4 Gradients

Because the objective contains $|W|$ and midpoint/radius terms, its gradient is not the standard backprop rule. Differentiating $\hat{z} = \mu W_3 + b_3 + s \odot (r|W_3|)$ and recursing gives, for each layer i ,

$$\frac{\partial \mathcal{L}}{\partial W_i} = \mu_i^\top g_i^\mu + (r_i^\top g_i^r) \odot \text{sign}(W_i), \quad g_{i-1}^\mu = g_i^\mu W_i^\top, \quad g_{i-1}^r = g_i^r |W_i|^\top, \quad (11)$$

with the ReLU step converting (g^μ, g^r) into gradients on $(z, \bar{z})$ via $\partial \mu / \partial z = \partial \mu / \partial \bar{z} = \frac{1}{2}$, $\partial r / \partial \bar{z} = -\partial r / \partial z = \frac{1}{2}$ and the masks $\mathbb{K}[z > 0], \mathbb{K}[\bar{z} > 0]$. The $\text{sign}(W_i)$ term is the interesting one: it is the gradient of the bound *radius*, and it is what actively drives the network towards small $|W|$ in the directions that widen the box. We validated the full derivation against central finite differences: the maximum relative error over 150 randomly probed parameters was 1.9×10^{-9} (python `tas_lib.py`).

6.5 Results

After 150 epochs (170 s) the certified model reaches **95.63%** clean test accuracy — statistically indistinguishable from the baseline’s 95.77% — and **82.89%** IBP-certified test accuracy at $\varepsilon = 0.1$ over the full 10 000-image test set. The absence of a clean-accuracy penalty is worth emphasising: at this ε and this network size, robustness is obtained essentially for free, which is not generally true at larger ε .

7 Step 5 — FGSM against the certified model

The assignment asks specifically for the previously fooled samples to be re-tested. Table 5 reports both that subset and the full test set.

	baseline	IBP-trained
<i>the 8685 test images that FGSM flipped in Step 2</i>		
clean accuracy	100.00	97.43
accuracy under FGSM re-crafted at $\varepsilon = 0.1$	0.00	90.13
accuracy on the <i>transferred</i> baseline adversarial images	—	93.10
accuracy under PGD-100	—	88.15
<i>full test set at $\varepsilon = 0.1$</i>		
clean	95.77	95.63
FGSM	8.92	88.02
PGD-50	3.13	86.28

Table 5: Step 5. Of the 8685 inputs that defeated the baseline, the certified model recovers **7828** (90.13%) even when the attack is regenerated against it; only 857 remain adversarial.

Two observations. First, the attack must be *re-crafted*: the old adversarial images transfer only partially (93.10% accuracy), so evaluating a defence on stale adversarial examples would overstate it — a classic pitfall. Second, the gap between FGSM (88.02%) and PGD (86.28%) on the certified model is only 1.7 points, versus 5.8 points on the baseline: gradient masking is not occurring, and FGSM is a reasonable proxy here. Figure 3 shows the full curves; the certified model degrades gracefully and only collapses well beyond the ε it was trained for.

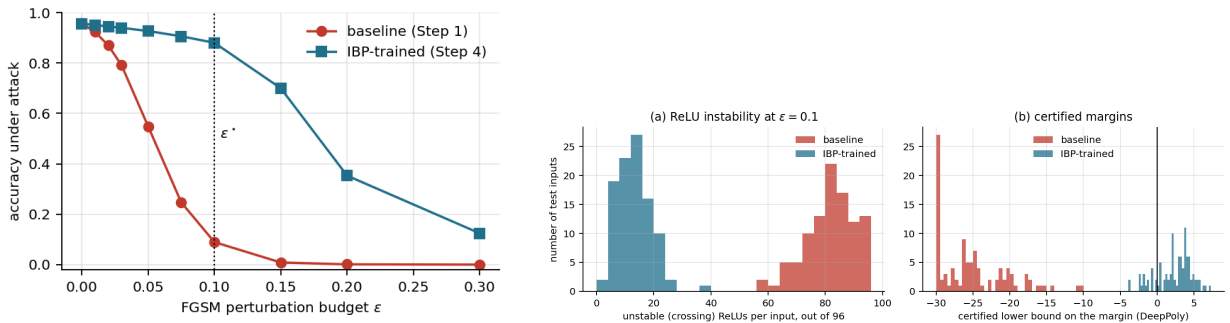


Figure 3: Left: accuracy under FGSM versus ε for both models. Right: (a) distribution of unstable ReLUs per input and (b) certified DeepPoly margins, at $\varepsilon = 0.1$. Certified training moves the margin distribution almost entirely to the right of zero and roughly halves the input’s exposure to ReLU branching.

8 Step 6 — Verifying the certified model, and why the result changes

Re-running the entire Step 3 pipeline on the new model gives the right-hand column of Table 4 and Figure 4. Four differences stand out, and they have a common cause.

1. Certification succeeds, and it is now *exact*. 82 of 100 inputs are certified, 17 are falsified by explicit counterexamples, 1 is misclassified: the 100 instances are fully partitioned with *no* timeouts. Because the MILP verifier is complete, the true robust accuracy of this model on this subset is exactly 82% — and DeepPoly reports exactly 82% as well. **The incompleteness gap has vanished entirely** (it was 34 points on the baseline at $\varepsilon = 0.03$). The remaining 5-point distance to PGD’s 87% is not verifier looseness; it is the difference between “PGD failed to find an

attack” and “an attack exists”, and the MILP settles it in favour of the latter for four inputs that PGD had missed. This is a good illustration of why an attack is not a substitute for a proof.

2. All three abstract domains agree. Interval, DeepZono and DeepPoly certify the identical set of 82 inputs. On the baseline they differed enormously (0% vs. 95% at $\varepsilon = 0.01$). The reason is direct: IBP training minimises the *box* bound, so the trained network is one on which the box abstraction happens to be tight, and the more expensive domains have nothing left to recover. Quantitatively, the median output-bound width is 3.76 for IBP and 3.68 for DeepPoly — a 2% improvement, against a $3.4\times$ improvement ($124.8 \rightarrow 36.5$) on the baseline. In practice this means the cheapest verifier (0.002 s for 100 inputs) is now as good as the $240\times$ more expensive one.

3. ReLU instability collapses. The mean number of unstable ReLUs per input drops from 81.7 to 12.8 out of 96 ($52.0 \rightarrow 7.4$ in layer 1, $29.7 \rightarrow 5.5$ in layer 2). This is the mechanism behind both previous points. Every crossing ReLU is a place where the relaxation must lose information and where the MILP must branch; the exact problem has $2^{\#\text{unstable}}$ activation patterns, so $2^{81.7}$ becomes $2^{12.8}$. Correspondingly, the MILP phase took 30.5 s and still timed out on the baseline, but resolved four inputs in 3.5 s on the certified model.

4. Certified training is a regulariser towards *verifiability*. Combining the above: IBP training does not only move the decision boundary away from the data, it reshapes the network so that a cheap over-approximation of it is accurate. The $\text{sign}(W)$ gradient term of Section 6 makes this explicit — it penalises exactly those weights that widen the interval, pushing pre-activations away from zero and stabilising ReLUs. The price is a loss of expressive capacity, which at $\varepsilon = 0.1$ on MNIST is small enough to be invisible ($95.77\% \rightarrow 95.63\%$) but becomes severe at larger ε or on harder datasets — the well-documented accuracy/robustness/verifiability trilemma.

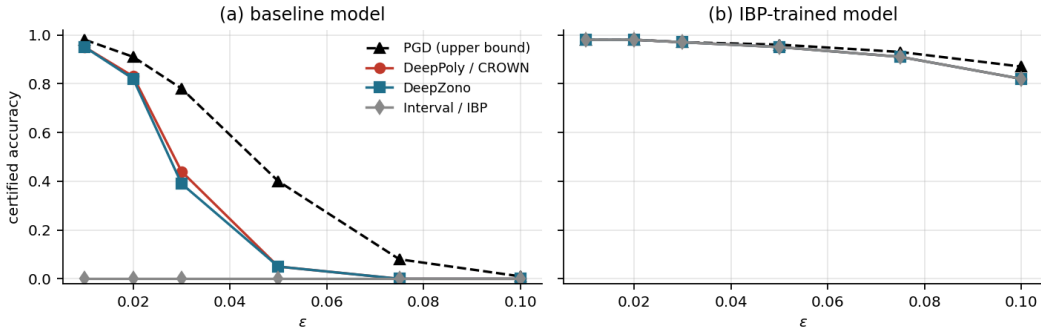


Figure 4: Certified accuracy versus ε . The shaded distance between the dashed PGD curve and the solid certified curves is the region of uncertainty: on the baseline (a) it is dominated by verifier incompleteness, on the certified model (b) it is only 5 points wide and, at $\varepsilon = 0.1$, entirely attributable to genuinely non-robust inputs rather than to the abstraction.

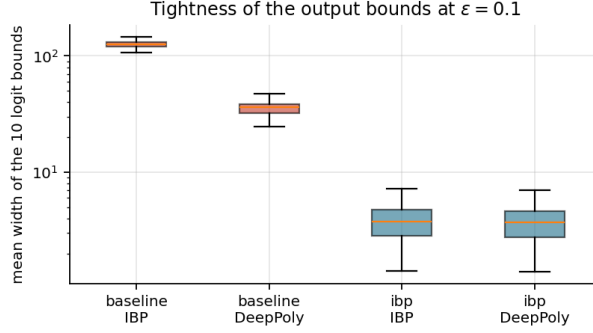


Figure 5: Width of the propagated logit bounds at $\varepsilon = 0.1$ (log scale). On the baseline, back-substitution (DeepPoly) buys a $3.4\times$ tightening over plain intervals; after certified training the two are within 2% of each other.

9 Discussion, limitations and conclusions

What was shown. A two-epoch MLP with 95.77% test accuracy is not merely attackable — it is *provably* non-robust: at $\varepsilon = 0.1$ a complete verifier finds an explicit counterexample for 98 of 100 test inputs, and no abstract domain can certify a single one. Replacing the point-wise loss with the worst-case-logit IBP loss of (8)–(9) raises FGSM accuracy from 8.92% to 88.02% and yields a machine-checked guarantee for 82% of inputs, at a clean-accuracy cost of 0.14 points.

Limitations. (i) Verification statistics are computed on the first 100 test images, the convention used by ERAN and CROWN, so they carry a sampling error of roughly ± 4 points; the IBP-certified accuracy over the full 10 000 images (82.89%) agrees closely with the 82% measured on the subset. (ii) One baseline instance is unresolved (30s MILP budget); a longer budget or LP-based bound tightening would close it. (iii) The guarantee is *local* — it says nothing about inputs outside the tested boxes — and it concerns ℓ_∞ perturbations only, not rotations, occlusions or distribution shift. (iv) We train at exactly the evaluation ε ; training at 1.1ε is known to buy a further point or two of certified accuracy. (v) The ERAN and `auto_LirPA` runs are provided as reproducible scripts and validated exports rather than executed here, for the environment reasons given in Section 2; our re-implementations were instead validated against an exact oracle, which is a stronger check than agreement with another incomplete tool.

Takeaway. The three families of techniques are complementary and should be reported together. An attack gives an upper bound on robustness, an abstract domain gives a lower bound, and only a complete method closes the interval. Certified training is what makes the lower bound worth computing: it converts a network on which the cheap analysis is useless into one on which the cheap analysis is exact.

10 Reproducibility

<code>code/tas_lib.py</code>	data, network, backprop, Adam, FGSM/PGD, IBP forward + gradients
<code>code/verifiers.py</code>	interval, DeepZono, DeepPoly/CROWN, MILP verifiers
<code>code/verification_driver.py</code>	the combined pipeline used by Steps 3 and 6
<code>code/step1..step6*.py</code>	the six steps of the assignment, in order
<code>code/run_all.py</code>	runs everything end-to-end
<code>code/export_models.py</code>	ONNX export (+ runtime check) and tool scripts
<code>code/to_pytorch.py</code>	converts the NumPy weights to a PyTorch <code>.pth</code>
<code>code/run_autolirpa.py,</code> <code>run_eran.sh</code>	drivers for the two official tools
<code>models/mlp_baseline.{npz,onnx}</code>	the Step-1 model
<code>models/mlp_ibp.{npz,onnx}</code>	the Step-4 model
<code>results/fooled_samples.npz</code>	the 8685 Step-2 adversarial samples
<code>results/step*.json,</code> <code>results/figures/</code>	all numbers and figures in this report

Table 6: Contents of the submission.

`pip install numpy scipy matplotlib onnx onnxruntime` then `cd code && python run_all.py` reproduces every number above (about 7 minutes on one CPU core; no GPU and no deep-learning framework required). Running the official tools additionally requires `pip install torch auto_LiRPA` (for `run_autolirpa.py`) or the `ethsri/eran:cpu` Docker image (for `run_eran.sh`).

References

- [1] I. Goodfellow, J. Shlens, C. Szegedy. *Explaining and Harnessing Adversarial Examples*. ICLR 2015.
- [2] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. ICLR 2018.
- [3] G. Singh, T. Gehr, M. Mirman, M. Püschel, M. Vechev. *Fast and Effective Robustness Certification*. NeurIPS 2018. (DeepZ / ERAN `deepzono`)
- [4] G. Singh, T. Gehr, M. Püschel, M. Vechev. *An Abstract Domain for Certifying Neural Networks*. POPL 2019. (DeepPoly / ERAN `deeppoly`)
- [5] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, L. Daniel. *Efficient Neural Network Robustness Certification with General Activation Functions*. NeurIPS 2018. (CROWN)
- [6] K. Xu et al. *Automatic Perturbation Analysis for Scalable Certified Robustness and Beyond*. NeurIPS 2020. (`auto_LiRPA`)
- [7] V. Tjeng, K. Xiao, R. Tedrake. *Evaluating Robustness of Neural Networks with Mixed Integer Programming*. ICLR 2019.
- [8] S. Gowal et al. *On the Effectiveness of Interval Bound Propagation for Training Verifiably Robust Models*. arXiv:1810.12715, 2018.
- [9] H. Zhang et al. *Towards Stable and Efficient Training of Verifiably Robust Neural Networks*. ICLR 2020. (CROWN-IBP)